Highlights

ParetoEnsembles.jl: A Julia Package for Multiobjective Parameter Estimation Using Pareto Optimal Ensemble Techniques

Jeffrey D. Varner

- Pareto ensemble method generates parameter sets spanning trade-off surfaces
- Incremental ranking reduces per-iteration cost from quadratic to linear
- Ensemble reveals parameter identifiability via compensatory correlations
- Held-out predictions within 8–9% on real coagulation experimental data
- Open-source Julia package with minimal dependencies and simple callback API

ParetoEnsembles.jl: A Julia Package for Multiobjective Parameter Estimation Using Pareto Optimal Ensemble Techniques

Jeffrey D. Varner[a,*]

[a] *Robert Frederick Smith School of Chemical and Biomolecular Engineering, Cornell University, Ithaca, NY 14853, USA*

Abstract

Background and Objective: Mathematical models of biological and engineering systems often have many adjustable parameters that must be estimated from multiple, potentially conflicting datasets. Rather than reporting a single best-fit parameter vector, it is often more informative to generate an *ensemble* of parameter sets that collectively map out the trade-offs among competing objectives. This paper presents ParetoEnsembles.jl, an open-source Julia package that generates such ensembles using Pareto Optimal Ensemble Techniques (POETs), a simulated-annealing-based algorithm that requires no gradient information and is defined entirely by four user-supplied callback functions.

Methods: The implementation corrects the original dominance relation from weak to strict Pareto dominance, reduces the per-iteration ranking cost from $O(n^2m)$ to $O(nm)$ through an incremental update scheme, and adds multi-chain parallel execution for improved front coverage. We demonstrate

*Corresponding author
Email address: `jdv27@cornell.edu` (Jeffrey D. Varner)

the package on a cell-free gene expression model fitted to experimental data and a blood coagulation cascade model with ten estimated rate constants and three objectives.

Results: A controlled synthetic-data study reveals parameter identifiability structure (individual rate constants off by several-fold yet model predictions accurate to 6–7%), and a five-replicate coverage analysis confirms that timing features (lag time, time-to-peak) are reliably covered while peak amplitude is systematically overconfident. Validation against published experimental thrombin generation data from a prothrombin titration study demonstrates that the ensemble predicts held-out conditions to within 1–10% despite inherent model approximation error.

Conclusions: ParetoEnsembles.jl provides uncertainty-quantified parameter ensembles that honestly reflect what the data can and cannot constrain, enabling calibrated predictions for downstream analyses including patient-specific simulations. The package is available under the MIT license at `https://github.com/varnerlab/ParetoEnsembles.jl`.
Keywords: multiobjective optimization, Pareto front, simulated annealing, ensemble methods, parameter estimation, uncertainty quantification

---

1. Introduction

Mathematical models of biological and engineering systems typically contain many adjustable parameters that must be estimated from experimental data. When multiple datasets measure different aspects of the same system, for example messenger RNA (mRNA) and protein concentrations in a gene expression circuit or yield and selectivity in a reactor, calibrating parameters

to one dataset often degrades the fit to another, giving rise to a multiobjective optimization (MOO) problem [1, 2, 3]. Compounding this difficulty, many models exhibit "sloppy" parameter sensitivities: broad, flat valleys in objective space where large regions of parameter space produce nearly equivalent fits [4]. In such settings, reporting a single best-fit parameter vector obscures the practical uncertainty; instead, one needs an *ensemble* of parameter sets that collectively map out the trade-offs among competing objectives. The boundary of this ensemble in objective space is the Pareto front, the set of solutions for which no objective can be improved without degrading at least one other, and the full cloud of near-optimal solutions surrounding it characterizes the range of model behaviors consistent with the available data.

Several families of algorithms have been developed to approximate Pareto fronts. Population-based evolutionary methods maintain a generation of candidate solutions that are selected, recombined, and mutated toward the front; prominent examples include the Non-dominated Sorting Genetic Algorithm II (NSGA-II) [5], its many-objective successor NSGA-III [6], and the Multiobjective Evolutionary Algorithm Based on Decomposition (MOEA/D), which decomposes the MOO problem into a set of scalar subproblems solved cooperatively [7]. An alternative trajectory-based strategy is multiobjective simulated annealing (SA), which extends the classical SA framework [8] to multiple objectives by using Pareto dominance to guide acceptance decisions; early formulations include the Pareto SA method of Czyżak and Jaszkiewicz [9] and the Archived Multiobjective Simulated Annealing (AMOSA) algorithm of Bandyopadhyay et al. [10]. Pareto Optimal Ensemble Techniques (POETs), introduced by Song et al. [11] and later implemented in the Julia program-

ming language as JuPOETs [12], belongs to this SA-based family; POETs ranks the current archive by counting how many other solutions dominate each candidate, then uses the rank as the energy in a Metropolis-type acceptance criterion. Mature software implementations of these and related algorithms are available in several languages, including pymoo in Python [13] and Metaheuristics.jl in Julia [14], both of which focus primarily on population-based evolutionary strategies. A complementary approach to characterizing parametric uncertainty is Bayesian inference, where Markov chain Monte Carlo (MCMC) or sequential Monte Carlo (SMC) methods sample from the posterior distribution over parameters given the data; widely used implementations include Stan [15], Turing.jl [16], and the ensemble MCMC sampler emcee [17]. While Bayesian methods provide a rigorous probabilistic interpretation of parameter uncertainty, they require specification of prior distributions and a scalar likelihood, scale poorly with the number of objectives, and can be prohibitively expensive for stiff ODE models; multiobjective ensemble methods such as POETs offer a lightweight alternative that naturally handles multiple competing objectives and returns a cloud of near-optimal solutions without requiring probabilistic modeling assumptions.

In this study, we present ParetoEnsembles.jl, a substantially rewritten Julia [18] implementation of the POETs algorithm. The package takes a user-defined model and a set of objective functions, each measuring how well a candidate parameter vector reproduces a particular dataset, and returns a collection of parameter sets spanning the trade-off surface, along with their objective values and dominance ranks. Several algorithmic corrections and performance improvements have been made relative to the original implemen-

tation: the dominance check has been corrected from weak to strict Pareto dominance, the per-iteration ranking cost has been reduced from $O(n^2m)$ to $O(nm)$ through an incremental update scheme (where n is the archive size and m is the number of objectives), and multi-chain parallel execution has been added to improve front coverage. We describe the methods underlying ParetoEnsembles.jl (Section 2), present results on two biological applications and ensemble-based downstream analyses (Section 3), discuss the findings in the context of the broader multiobjective optimization landscape (Section 4), and conclude with a summary and outlook (Section 5). Validation on standard benchmarks, a comparison with NSGA-II, hyperparameter sensitivity analysis, and software implementation details are provided in the Supplementary Material.

2. Methods

2.1. Pareto dominance and ranking

Consider an archive of n solutions evaluated on m objectives, where lower values are preferred. Solution $\mathbf{x}_j$ *strictly dominates* solution $\mathbf{x}_i$, written $\mathbf{x}_j \prec \mathbf{x}_i$, if and only if $f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i)$ for all $k \in \{1, \ldots, m\}$ and there exists at least one objective k for which $f_k(\mathbf{x}_j) < f_k(\mathbf{x}_i)$ (Equation (1)). The *Pareto rank* of solution $\mathbf{x}_i$ is the number of archive members that strictly dominate it (Equation (2)), so that solutions with rank zero lie on the Pareto front and identical solutions, which do not dominate each other under strict dominance, share the same rank. It is worth noting that the original POETs implementation [11, 12] used *weak* dominance, checking only that $f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i)$ for all k without requiring strict inequality in at least one objective;

under weak dominance, identical solutions count as dominating each other, which inflates their ranks and causes excessive pruning of good solutions from the archive. The current implementation enforces strict Pareto dominance, and the pairwise test iterates over objectives with an early-exit condition that terminates as soon as any objective of j is found to be worse than i, avoiding unnecessary comparisons.

$$f_k(\mathbf{x}_j) \leq f_k(\mathbf{x}_i) \quad \forall\, k \in \{1, \ldots, m\} \qquad \text{and} \qquad \exists\, k : f_k(\mathbf{x}_j) < f_k(\mathbf{x}_i). \tag{1}$$

$$R(\mathbf{x}_i) = \big|\{\, j \neq i \mid \mathbf{x}_j \prec \mathbf{x}_i \,\}\big|. \tag{2}$$

2.2. Incremental ranking

A central computational concern is the cost of ranking. When a single candidate solution $\mathbf{x}_{\text{new}}$ is appended to an archive of $n-1$ existing solutions with known ranks $\mathbf{R}$, the naïve approach recomputes all $n(n-1)$ pairwise dominance checks at $O(n^2 m)$ cost. Because only one solution changed, however, we can instead perform a single $O(nm)$ pass over the existing archive: for each existing solution $\mathbf{x}_i$, we test dominance in both directions, incrementing R_i if the new solution dominates $\mathbf{x}_i$ and accumulating toward the new solution's rank if $\mathbf{x}_i$ dominates it. This incremental procedure reduces the per-iteration cost from $O(n^2 m)$ to $O(nm)$, with a full $O(n'^2 m)$ re-rank performed only after archive pruning when n' is small.

2.3. Pareto simulated annealing

The complete Pareto simulated annealing procedure combines the dominance check and incremental ranking into a single outer-inner loop structure. The algorithm is seeded with an initial parameter vector $\mathbf{x}_0$, an initial

temperature $T = 1$, and an archive containing the single evaluated solution $(\mathbf{x}_0, \mathbf{f}(\mathbf{x}_0))$. At each temperature level, N_{iter} candidate solutions are generated by perturbing the current best parameter vector through a user-supplied neighbor function, and the rank array is updated incrementally at $O(nm)$ cost per candidate. The acceptance probability, computed from the rank array and the current temperature, determines whether the candidate is retained; upon acceptance, solutions whose rank meets or exceeds the cutoff R_{cutoff} are pruned and a full re-rank is performed on the smaller remaining set, with a hard size cap $n_{\max}$ enforced if the archive still exceeds the limit. Upon rejection, the candidate is immediately removed and the rank array is restored from a saved snapshot, a strategy we refer to as *pop-on-reject*; in the original implementation, rejected candidates remained in the archive, causing unbounded growth of n between accepted moves and progressively more expensive ranking operations, and the pop-on-reject strategy eliminates this problem entirely. To improve front coverage, multiple independent SA chains can be run from different starting points and their archives merged, with a final re-rank performed on the combined set. This multi-chain strategy provides near-linear parallel speedup and ensures diverse exploration of the trade-off surface. Pseudocode for all algorithmic components, software design details, and the public API are provided in the Supplementary Material (Sections S1 and S2).

3. Results

We first validated ParetoEnsembles.jl on two standard multiobjective benchmarks, the constrained Binh–Korn problem [19] and the unconstrained

Fonseca–Fleming problem [20], recovering the known Pareto fronts in both cases (Section S4). A head-to-head comparison with NSGA-II [5] at matched evaluation budgets (∼110,000) showed that ParetoEnsembles.jl achieves comparable hypervolume and 5–7× lower inverted generational distance while producing 10–30× denser fronts (Section S5), and a hyperparameter sensitivity analysis confirmed robustness to the choice of rank cutoff, cooling rate, and iterations per temperature (Section S6). We focus here on two biological applications that illustrate the practical value of ensemble-based parameter estimation.

3.1. Cell-free gene expression

As a first biological application, we considered the estimation of kinetic parameters for a cell-free gene expression circuit describing the production of deGFP protein driven by the endogenous sigma factor σ_{70} via the P70 promoter, following the effective biophysical framework of Adhikari et al. [21]. The system is governed by two ordinary differential equations for mRNA (m) and protein (p), where $\dot{m} = \alpha\, u(\sigma_{70}) - \delta_m\, m$ and $\dot{p} = \kappa\, m\, w(t) - \delta_p\, p$, with $u(\sigma_{70}) = \sigma_{70}^n/(K^n + \sigma_{70}^n)$ representing a Hill-type promoter activity function and $w(t) = \exp(-\ln 2 \cdot t/\tau_{1/2})$ capturing the experimentally observed decay of translational capacity over the course of the cell-free reaction. Five parameters were treated as unknown (the maximum transcription rate α, the effective translation rate constant κ, the mRNA degradation rate δ_m, the promoter dissociation constant K, and the translation capacity half-life $\tau_{1/2}$) while the remaining parameters ($\sigma_{70} = 35$ nM, $n = 1.5$, $\delta_p = 0.005$ h^{-1}) were fixed at values taken from the literature.

We fitted the model to experimental data from Adhikari et al. [21],

consisting of mRNA and deGFP protein concentrations measured at five time points (0, 2, 4, 8, and 16 h) in triplicate, with reported means and standard deviations. Two objectives were defined as standard-deviation-weighted sums of squared errors: $\varepsilon_{\mathrm{mRNA}} = \sum_i [(m_i^{\mathrm{sim}} - m_i^{\mathrm{obs}})/\sigma_i^m]^2$ and $\varepsilon_{\mathrm{protein}} = \sum_i [(p_i^{\mathrm{sim}} - p_i^{\mathrm{obs}})/\sigma_i^p]^2$, and we ran ten parallel chains with $R_{\mathrm{cutoff}} = 8$ and $N_{\mathrm{iter}} = 50$, starting from random initial parameter vectors drawn uniformly within the bounds. The resulting ensemble captured the dynamics of both mRNA and protein expression (Figure 1a,b): the 95% confidence interval of the ensemble simulations bracketed the experimental data at all time points, and the ensemble mean tracked the observed trajectories. The Pareto front revealed a clear trade-off between fitting the mRNA and protein time courses (Figure 1c), with parameter sets that minimized $\varepsilon_{\mathrm{mRNA}}$ tending to have slightly larger $\varepsilon_{\mathrm{protein}}$ and vice versa. This trade-off arises naturally because the mRNA degradation rate and translation capacity half-life have opposing effects on the two species, so that the ensemble provides a family of plausible parameter sets rather than a single point estimate, enabling downstream uncertainty quantification of model predictions.

3.2. Blood coagulation cascade

To demonstrate the package on a larger-scale system and to explore the role of parameter ensembles in uncertainty quantification, we considered the Hockin–Mann model of tissue-factor-initiated blood coagulation [22], a mechanistic ODE model describing 34 chemical species interacting through 27 reactions governed by 42 rate constants that captures the initiation, amplification, and inhibition phases of thrombin generation. All rate constants are available in the literature and were used as ground truth, and we selected ten

key catalytic and inhibition rate constants for estimation spanning the extrinsic factor Xase (k_{cat}), intrinsic factor Xase (k_{cat}), prothrombinase (k_{cat}), and antithrombin III inhibition pathways. Because these rate constants span six orders of magnitude (from $\sim$1 to $\sim 10^7$), we worked in log-space with bounds of ± 1.5 orders of magnitude around the literature values, and generated noisy synthetic thrombin generation assay (TGA) data by simulating the model with the true rate constants at three tissue factor concentrations (5, 15, and 25 pM) and adding 15% multiplicative Gaussian noise to the total thrombin trajectory at each condition. Three purely data-driven objectives were defined, one for the normalized sum-of-squared error of the thrombin trajectory at each of the three TF concentrations, with no regularization term, so that parameter recovery reflects only what the training data can constrain rather than being aided by a penalty that pulls estimates toward the known truth.

We ran eight parallel chains with $R_{\text{cutoff}} = 10$, $N_{\text{iter}} = 40$, and $\alpha = 0.92$, and the resulting ensemble of 222 parameter sets (rank ≤ 1) produced thrombin trajectories that closely matched the noisy training data at all three TF concentrations (Figure 2a), with 95% confidence intervals that bracketed the data throughout the time course. Parameter recovery was mixed, revealing the practical identifiability structure of the system (Figure 2b): some rate constants were well recovered (the IIa+ATIII inhibition rate to within 2% and the Factor Va activation rate to within 5%) while others were poorly constrained by thrombin data alone, with the intrinsic factor Xase k_{cat} off by a factor of five and the direct Xa-mediated thrombin generation rate ($k_{\text{Xa}\to\text{IIa}}$) overestimated by more than twofold. The three-dimensional

Pareto front (Figure 2c) revealed the expected trade-offs among fitting the three TF conditions, demonstrating that ParetoEnsembles.jl can handle systems of moderate complexity with tens of species, ten or more estimated parameters, and three competing objectives. That several parameters are poorly recovered while the model still fits the training data well is itself a central finding: it reflects the sloppy sensitivity structure of the coagulation cascade, where compensatory adjustments among rate constants can produce nearly identical thrombin trajectories from very different parameter vectors, and it motivates the ensemble-based downstream analyses described next.

3.3. Ensemble-based prediction and identifiability analysis

A key advantage of generating parameter ensembles rather than single point estimates is the ability to propagate uncertainty forward through the model to obtain prediction intervals whose width reflects what the training data can and cannot constrain. To test this, we used the coagulation ensemble, which was trained only on TGA curves at 5, 15, and 25 pM TF, to predict the full thrombin time course at three held-out tissue factor concentrations (10, 20, and 30 pM) that were never used during optimization. The ensemble mean tracked the true trajectories at all three conditions, predicting peak thrombin to within 6–7% of the true values, and the 95% prediction intervals captured the overall shape of the thrombin generation curve (Figure 3a). However, the prediction bands did not fully bracket the true trajectories at the peak, revealing a systematic positive bias in peak amplitude that the ensemble's uncertainty does not account for. This finding would be entirely invisible with a single best-fit parameter vector, which provides a point prediction but no indication of where or how much to trust it. To quantify this

further, we extracted three clinically relevant TGA features (lag time, peak thrombin concentration, and endogenous thrombin potential (ETP)) from both the ensemble predictions and the true trajectories at each held-out condition (Figure 3d). Lag time was well predicted with appropriately wide 95% intervals that covered the true value at all three concentrations, while peak thrombin and ETP showed a consistent 4–7% overprediction with intervals too tight to achieve coverage, indicating that the ensemble is slightly overconfident about thrombin amplitude even as it correctly captures the timing of the coagulation response.

The ensemble also reveals the identifiability structure of the estimated parameters through their pairwise correlations (Figure 3b). The correlation matrix is strikingly rich in structure: the strongest negative correlation was between the extrinsic factor Xase k_{cat} and the prothrombinase k_{cat} ($r = -0.81$), indicating that these two catalytic rates compensate each other so that if the initiation-phase enzyme is faster, the amplification-phase enzyme must be correspondingly slower to produce the same thrombin trajectory. Similarly strong compensatory relationships were observed between the direct Xa-mediated thrombin generation rate and Factor Va activation ($r = -0.93$), reflecting the competition between direct and prothrombinase-mediated pathways, and between the TF=VIIa→IX catalytic rate and the mIIa→IIa conversion rate ($r = -0.73$), coupling the initiation and propagation phases. These correlations explain why individual parameters can be off by several-fold while the model predictions remain accurate to within a few percent: the ensemble captures the compensatory manifold in parameter space along which the data are uninformative, and it is precisely this

manifold that determines the width and bias of the prediction intervals.

Finally, we used the ensemble to simulate a clinically relevant perturbation: Factor VIII deficiency (hemophilia A) at varying severity levels. By reducing the initial Factor VIII concentration to 30% (mild hemophilia) and 5% (severe hemophilia) of the nominal plasma level while keeping all estimated rate constants at their ensemble values, we obtained patient-specific predictions of thrombin generation that captured both the expected dose-dependent reduction in peak thrombin and the associated uncertainty (Figure 3c). Peak thrombin decreased from 684 ± 22 nM in normal plasma to 563 ± 22 nM for mild hemophilia and 401 ± 21 nM for severe hemophilia, with the true trajectories falling within the ensemble prediction bands for all three conditions. The prediction intervals widened slightly as Factor VIII was reduced, consistent with the expectation that the model becomes less constrained as the system is driven further from the conditions used for training, and the fact that the ensemble correctly brackets the truth for these out-of-distribution perturbations, even though several individual rate constants are poorly recovered, underscores the value of ensemble-based uncertainty quantification for translational applications where model predictions inform clinical decisions.

3.4. Validation against experimental data

The coagulation analyses presented above use synthetic data where the ground truth is known, enabling rigorous assessment of parameter recovery and coverage but leaving open the question of whether the ensemble approach works when the model is an approximation of reality rather than the data-generating process itself. To address this, we fitted the Hockin–

Mann model to published experimental data from Butenas et al. [23], who measured thrombin generation in a reconstituted synthetic plasma system initiated by 5 pmol/L tissue factor–VIIa complex on phospholipid vesicles at five prothrombin concentrations ranging from 50% to 150% of the mean plasma value, with all other coagulation factors held at their nominal concentrations (thrombin time-course data were digitized from Figure 3 of [23], with peak values calibrated against the quantitative ratios reported in Table 2 of the same study). This system provides an ideal test case because the initial conditions are precisely known (Table 1 of [23]), the protein C pathway is absent (matching the basic Hockin–Mann model), and the experimental data span a physiologically relevant range of prothrombin levels that is directly analogous to interindividual variability in clinical populations.

We defined three training objectives as the normalized SSE between simulated and experimental thrombin time courses at 50%, 100%, and 150% of normal prothrombin, and held out the 75% and 125% conditions for validation. Eight parallel chains were run with the same hyperparameters as the synthetic study, yielding an ensemble of 390 parameter sets (rank ≤ 1). The ensemble captured the experimental thrombin profiles at the training conditions (Figure 4a), with peak thrombin predicted to within 0.1–10% across the three prothrombin levels; the residual discrepancies reflect genuine model–data tension in which the Hockin–Mann model's sensitivity to prothrombin concentration does not perfectly match the experimental system, a tension that is visible in the trade-off structure of the Pareto front (Figure 4c). At the held-out validation conditions, the ensemble predicted peak thrombin at 75% and 125% prothrombin to within 1–10% of the experimental values

(Figure 4b), with the ensemble mean tracking the data closely and the 95% prediction intervals bracketing most of the experimental time points. The estimated rate constants deviated from their nominal literature values (Figure 4d), as expected when fitting real data where the model structure is only an approximation of the underlying biochemistry; these deviations represent the compensatory adjustments that the model makes to accommodate experimental reality, and the spread of the ensemble around each parameter reflects the degree to which the data constrain that rate constant given the model's structural limitations.

4. Discussion

ParetoEnsembles.jl provides a lightweight Julia implementation of Pareto Optimal Ensemble Techniques for multiobjective optimization that addresses several limitations of the original POETs algorithm [11, 12]. The current version corrects the dominance relation from weak to strict, reduces the per-iteration ranking cost from $O(n^2m)$ to $O(nm)$ through incremental updating, bounds archive growth through the pop-on-reject strategy and a hard size cap, adds both threaded ranking and multi-chain parallel execution, and provides built-in hypervolume computation and convergence diagnostics. The callback-based design decouples the optimization algorithm from the problem definition, allowing users to apply ParetoEnsembles.jl to any domain where objective functions can be evaluated pointwise. On standard benchmark problems, ParetoEnsembles.jl produces fronts with hypervolume and inverted generational distance competitive with or superior to NSGA-II at matched evaluation budgets, while retaining ensembles that are 10–30× denser than

those returned by a leading evolutionary method (Section S5). This density difference is not merely cosmetic: meaningful computation of prediction intervals, parameter correlations, and coverage statistics from an ensemble requires hundreds to thousands of members that fill the near-optimal region of parameter space, not just the 100–200 non-dominated solutions typical of a population-based method, and the downstream analyses in this paper would not have been possible with the sparse fronts returned by NSGA-II. While NSGA-II is considerably faster in wall time, reflecting the inherently sequential nature of SA chains versus the population-parallel evaluation of evolutionary methods, ParetoEnsembles.jl targets problems where the primary goal is to characterize the *cloud* of near-optimal parameter sets, not just the front itself, and where the simplicity of the interface lowers the barrier to generating parameter ensembles as part of a modeling workflow.

The biological applications presented here illustrate both the strengths and the current scope of the package, and they highlight why ensemble-based parameter estimation is qualitatively different from, and more informative than, conventional single-point optimization. The cell-free gene expression example demonstrates that even a small model with five parameters and two objectives can reveal meaningful trade-offs when fitted to real experimental data, and the coagulation cascade example shows that the approach scales to systems with 34 species, ten estimated parameters spanning six orders of magnitude, and three competing objectives. A central finding of the coagulation study is that accurate model predictions do not require accurate parameter recovery: the ensemble predicted thrombin generation at held-out TF concentrations to within 6–7% of the true peak even though

individual rate constants were off by as much as fivefold, because the pairwise parameter correlations revealed extensive compensatory structure (the extrinsic Xase and prothrombinase catalytic rates, for example, trade off at $r = -0.81$) so that many different parameter combinations produce nearly identical thrombin trajectories. This compensatory manifold is invisible to single-point estimation methods, which return one parameter vector with no indication of how tightly the data constrain it, and it is precisely the manifold that determines whether a model's predictions can be trusted when extrapolated to new conditions. The held-out validation also exposed an honest limitation: while the ensemble correctly predicted the timing of thrombin generation (lag time covered at all conditions), it showed a systematic positive bias in peak amplitude and ETP with confidence intervals that were too tight to achieve nominal coverage, suggesting that additional data types, such as measurements of upstream intermediates or different experimental readouts, would be needed to fully constrain the amplitude of the thrombin burst. Because the coagulation study uses synthetic data generated from the same model used for fitting, it is important to clarify what this controlled experiment can and cannot demonstrate: it validates the algorithm's ability to explore the trade-off surface and provides a ground-truth benchmark for assessing coverage and identifiability, but it does not test robustness to model misspecification. To address this, we repeated the study with six fixed rate constants perturbed by $\pm 30\%$ so that the fitting model was genuinely misspecified (Section S7); the ensemble still predicted held-out peak thrombin to within 4–6%, though trajectory-level coverage degraded to 31–48%, confirming that the approach is robust to moderate structural error while

revealing that parametric uncertainty quantification alone cannot account for model inadequacy. The hemophilia A simulations demonstrated that the ensemble can propagate parametric uncertainty through clinically relevant perturbations, with prediction intervals that widen as the system is driven further from training conditions and that correctly bracket the true trajectories even when the underlying parameters are individually poorly identified. These capabilities illustrate why ensemble methods are essential in applications where model predictions inform consequential decisions, and they provide a template for how ParetoEnsembles.jl can be used in practice. The hyperparameter sensitivity analysis (Section S6) further shows that the algorithm is robust to the choice of rank cutoff, cooling rate, and iterations per temperature, so that users can adopt the default settings without extensive tuning, and the convergence trace feature provides a built-in diagnostic for monitoring whether the annealing schedule was sufficient.

It is important to distinguish the ensemble produced by ParetoEnsembles.jl from a Bayesian posterior distribution. The ensemble is a collection of parameter vectors that are near-optimal with respect to multiple objectives, selected by simulated annealing with rank-based acceptance; it does not correspond to samples from a probability distribution conditioned on the data, and the 95% intervals reported throughout this paper are empirical quantiles of the ensemble predictions, not Bayesian credible intervals. In particular, the ensemble makes no claim about the relative probability of different parameter vectors, only that they produce comparably good fits, and its coverage properties depend on how thoroughly the simulated annealing chains explore the near-optimal region rather than on convergence to a stationary distribu-

tion. Bayesian approaches such as MCMC [15, 16] or ensemble MCMC [17] provide a principled probabilistic framework for parameter uncertainty, but they require specification of prior distributions and a scalar likelihood function, which is not naturally defined when multiple incommensurable objectives must be balanced simultaneously; they also face well-known challenges with multimodal posteriors and stiff ODE likelihoods that can make convergence slow or unreliable. The Pareto ensemble approach sidesteps these difficulties at the cost of probabilistic interpretability, and users who require calibrated posterior credible intervals should consider Bayesian methods as a complementary analysis. To verify that the coverage patterns are systematic rather than artifacts of a single noise realization, we repeated the synthetic coagulation study five times with independent noise seeds and computed feature-level coverage rates across replicates. The results confirmed a clear and reproducible pattern: lag time and time-to-peak achieved 100% coverage (5/5 replicates, all held-out conditions), while peak thrombin was covered in only 40% of replicates and ETP in 60%, with trajectory-level coverage averaging 87–90% across conditions. The mean held-out peak error was $7.1 \pm 5.0\%$ (range 0.9–17.9%), confirming that the single-replicate results reported above are representative. This systematic pattern, in which timing is well-calibrated but amplitude is overconfident, likely reflects the concentration of the ensemble along the compensatory manifold identified by the correlation analysis: the simulated annealing chains efficiently explore directions in parameter space along which objective values change (i.e., along the Pareto front), but they may underexplore directions orthogonal to the front where objective values are nearly constant yet model predictions dif-

fer, leading to prediction intervals that are well-calibrated for some outputs but overconfident for others. We also verified that the downstream predictions are insensitive to the choice of which archive members are included in the ensemble: sweeping the rank cutoff from 0 (Pareto front only) through rank ≤ 5 changed the mean held-out peak error by less than 0.2 percentage points, confirming that the near-optimal solutions surrounding the front carry essentially the same predictive information as the front itself.

Several features are deliberately omitted from the current scope. The package does not implement gradient-based refinement, surrogate modeling, or adaptive cooling schedules, nor does it provide built-in visualization or benchmark problem libraries. Integration with Julia's automatic differentiation ecosystem, for example through ForwardDiff.jl or Enzyme.jl, could enable hybrid gradient-free/gradient-based strategies for problems where gradients are available for some but not all objectives, and adaptive archive management using hypervolume indicators [24] in place of simple rank-based pruning would provide more principled control over the distribution of retained solutions along the Pareto front.

5. Conclusions

We have presented ParetoEnsembles.jl, a Julia package for generating Pareto-optimal parameter ensembles using simulated annealing with incremental dominance ranking. The package corrects a longstanding weak dominance bug in the original POETs implementation, reduces the per-iteration ranking cost from quadratic to linear in the archive size, and adds multi-chain parallel execution that provides improved front coverage from diverse start-

ing points. The package was applied to two biological systems of increasing complexity: a cell-free gene expression circuit fitted to experimental mRNA and protein time-course data, and the Hockin–Mann blood coagulation cascade model with 34 species and ten estimated rate constants spanning six orders of magnitude. A controlled synthetic-data study revealed that accurate model predictions do not require accurate parameter recovery, as the ensemble predicted held-out thrombin generation to within 6–7% despite individual parameters being off by several-fold, because the ensemble captures the compensatory manifold along which the training data are uninformative; the identifiability analysis, patient-specific hemophilia A simulations, and model misspecification study collectively demonstrate that ensemble-based uncertainty quantification honestly reflects what the data can and cannot constrain. Validation against published experimental thrombin generation data from Butenas et al. [23] confirmed that the approach generalizes beyond the synthetic setting: the ensemble trained on three prothrombin levels predicted held-out conditions to within 1–10%, with honest uncertainty bands, despite the inherent model approximation error that is absent from synthetic studies. ParetoEnsembles.jl is open source, registered in the Julia General registry, and can be installed with a single command; we anticipate that its minimal interface and dependency-free design will make it a useful tool for researchers who need to generate parameter ensembles for mechanistic models in systems biology and beyond.

Acknowledgments

J.D.V. acknowledges support from the National Heart, Lung, and Blood Institute (NHLBI) of the National Institutes of Health (NIH) under awards R33 HL141787 and R01 HL71944.

Author Contributions

J.D.V. conceived the study, developed the software, performed all analyses, and wrote the manuscript.

Competing Interests

The author declares no competing interests.

Data and Code Availability

ParetoEnsembles.jl is open source under the MIT license. The source code is available at `https://github.com/varnerlab/ParetoEnsembles.jl`, and documentation is hosted at `https://varnerlab.github.io/ParetoEnsembles.jl/dev/`. The package is registered in the Julia General registry and can be installed via `Pkg.add("ParetoEnsembles")`. Example code reproducing all results in this paper is included in the `paper/code` directory of the source repository.

References

[1] K. Miettinen, Nonlinear Multiobjective Optimization, Springer, 1999. `doi:10.1007/978-1-4615-5563-6`.

[2] K. Deb, Multi-Objective Optimization Using Evolutionary Algorithms, John Wiley & Sons, 2001.

[3] C. A. Coello Coello, G. B. Lamont, D. A. Van Veldhuizen, Evolutionary Algorithms for Solving Multi-Objective Problems, 2nd Edition, Springer, 2007. doi:10.1007/978-0-387-36797-2.

[4] R. N. Gutenkunst, J. J. Waterfall, F. P. Casey, K. S. Brown, C. R. Myers, J. P. Sethna, Universally sloppy parameter sensitivities in systems biology models, PLoS Computational Biology 3 (10) (2007) e189. doi:10.1371/journal.pcbi.0030189.

[5] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Transactions on Evolutionary Computation 6 (2) (2002) 182–197. doi:10.1109/4235.996017.

[6] K. Deb, H. Jain, An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints, IEEE Transactions on Evolutionary Computation 18 (4) (2014) 577–601. doi:10.1109/TEVC.2013.2281535.

[7] Q. Zhang, H. Li, MOEA/D: A multiobjective evolutionary algorithm based on decomposition, IEEE Transactions on Evolutionary Computation 11 (6) (2007) 712–731. doi:10.1109/TEVC.2007.892759.

[8] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by simulated annealing, Science 220 (4598) (1983) 671–680. doi:10.1126/science.220.4598.671.

[9] P. Czyżak, A. Jaszkiewicz, Pareto simulated annealing—a metaheuristic technique for multiple-objective combinatorial optimization, Journal of Multi-Criteria Decision Analysis 7 (1) (1998) 34–47. doi:10.1002/(SICI)1099-1360(199801)7:1<34::AID-MCDA161>3.0.CO;2-6.

[10] S. Bandyopadhyay, S. Saha, U. Maulik, K. Deb, A simulated annealing-based multiobjective optimization algorithm: AMOSA, IEEE Transactions on Evolutionary Computation 12 (3) (2008) 269–283. doi:10.1109/TEVC.2007.900837.

[11] S. Song, A. Chakrabarti, J. Bhatt, D. Bhatt, J. D. Varner, Identifying ensembles of signal transduction models using Pareto optimal ensemble techniques (POETs), Biotechnology Journal 5 (7) (2010) 768–780. doi:10.1002/biot.201000059.

[12] D. Bassen, M. Vilkhovoy, M. Minot, J. Butcher, J. D. Varner, JuPOETs: A constrained multiobjective optimization approach to estimate biochemical model ensembles in the Julia programming language, BMC Systems Biology 11 (1) (2017) 10. doi:10.1186/s12918-016-0380-2.

[13] J. Blank, K. Deb, pymoo: Multi-objective optimization in Python, IEEE Access 8 (2020) 89497–89509. doi:10.1109/ACCESS.2020.2990567.

[14] J.-A. Mejía-de Dios, E. Mezura-Montes, C. A. Coello Coello, Metaheuristics.jl: Towards reproducible optimization research, Journal of Open Source Software 7 (78) (2022) 4723. doi:10.21105/joss.04723.

[15] B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, A. Riddell, Stan: A probabilistic

programming language, Journal of Statistical Software 76 (1) (2017) 1–32. doi:10.18637/jss.v076.i01.

[16] H. Ge, K. Xu, Z. Ghahramani, Turing: A language for flexible probabilistic inference, in: Proceedings of the 21st International Conference on Artificial Intelligence and Statistics, 2018, pp. 1682–1690.

[17] D. Foreman-Mackey, D. W. Hogg, D. Lang, J. Goodman, emcee: The MCMC hammer, Publications of the Astronomical Society of the Pacific 125 (925) (2013) 306–312. doi:10.1086/670067.

[18] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.

[19] T. T. Binh, U. Korn, MOBES: A multiobjective evolution strategy for constrained optimization problems, in: Proceedings of the Third International Conference on Genetic Algorithms (Mendel 97), 1997, pp. 176–182.

[20] C. M. Fonseca, P. J. Fleming, An overview of evolutionary algorithms in multiobjective optimization, Evolutionary Computation 3 (1) (1995) 1–16. doi:10.1162/evco.1995.3.1.1.

[21] A. Adhikari, M. Vilkhovoy, S. Vadhin, H. E. Lim, J. D. Varner, Effective biophysical modeling of cell free transcription and translation processes, Frontiers in Bioengineering and Biotechnology 8 (2020) 539081. doi:10.3389/fbioe.2020.539081.

[22] M. F. Hockin, K. C. Jones, S. J. Everse, K. G. Mann, A model for the stoichiometric regulation of blood coagulation, Journal of Biological Chemistry 277 (21) (2002) 18322–18333. doi:10.1074/jbc.M201173200.

[23] S. Butenas, C. van't Veer, K. G. Mann, "normal" thrombin generation, Blood 94 (7) (1999) 2169–2178. doi:10.1182/blood.V94.7.2169.419k22_2169_2178.

[24] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, V. G. da Fonseca, Performance assessment of multiobjective optimizers: An analysis and review, IEEE Transactions on Evolutionary Computation 7 (2) (2003) 117–132. doi:10.1109/TEVC.2003.810758.

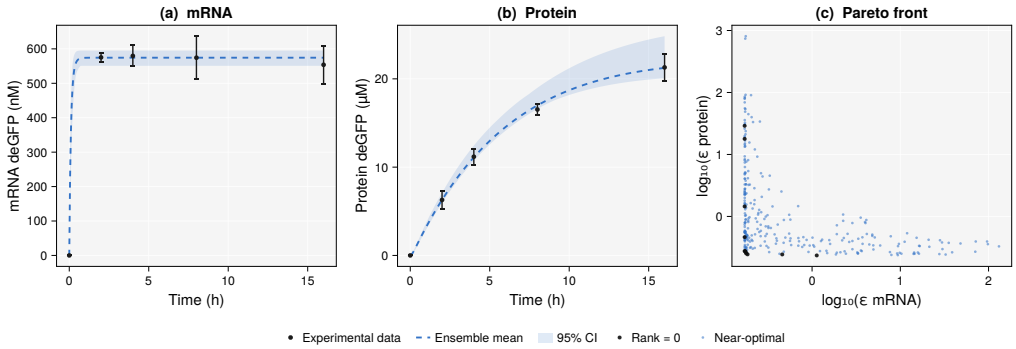

Figure 1: Ensemble estimation for a cell-free gene expression circuit ($\sigma_{70} \rightarrow$ P70 $\rightarrow$ deGFP) fitted to experimental data from Adhikari et al. [21]. (a) mRNA and (b) protein concentration versus time; the blue dashed line is the ensemble mean, the shaded region is the 95% confidence interval, and black points are experimental data with error bars. (c) Pareto front showing the trade-off between mRNA and protein fitting error.

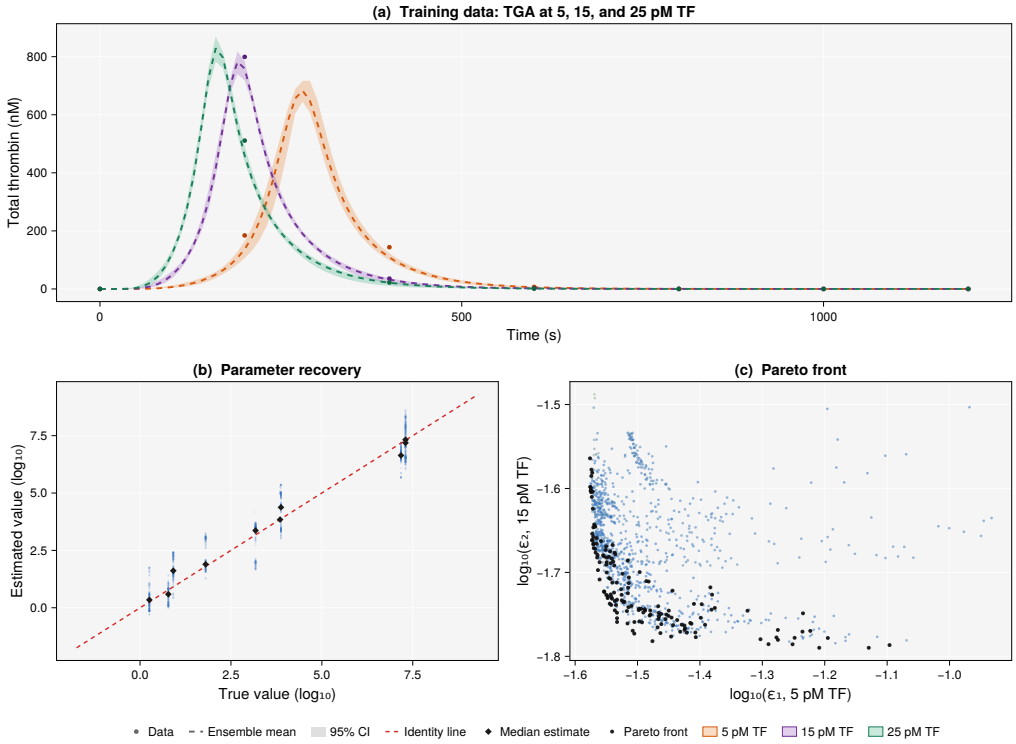

Figure 2: Training results for the Hockin–Mann blood coagulation model (34 species, 10 estimated rate constants, 3 data-driven objectives, no regularization). (a) Total thrombin concentration versus time at 5 pM (amber), 15 pM (purple), and 25 pM (teal) tissue factor; shaded regions are the ensemble 95% CI, dashed lines are the ensemble mean, and points are noisy synthetic data (15% CV). (b) Parameter recovery in log-space; the dashed red line is the identity and diamonds show the median ensemble estimate; scatter around each parameter reveals the degree of identifiability from thrombin data alone. (c) Pareto front projection (ε_1 vs. ε_2, colored by ε_3), with rank-zero solutions in black.

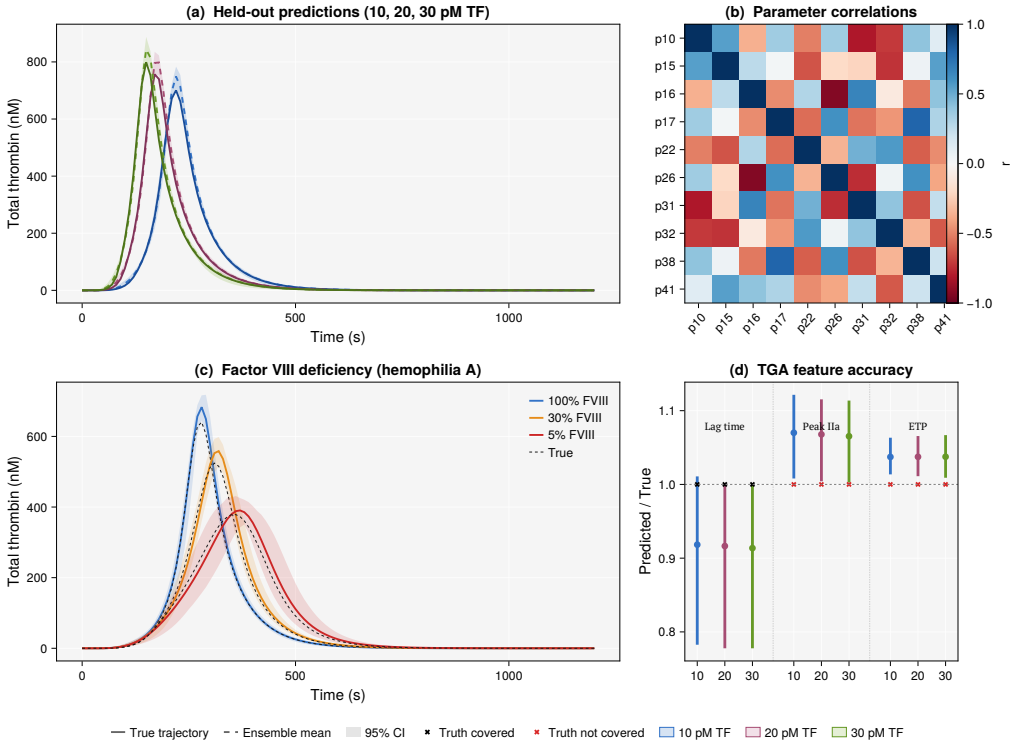

Figure 3: Ensemble-based uncertainty quantification for the coagulation model. (a) Held-out thrombin predictions at 10, 20, and 30 pM TF (not used during training); solid lines are true trajectories, dashed lines are ensemble means, and shaded regions are 95% CIs. The ensemble captures the overall shape but shows a systematic positive bias at peak thrombin. (b) Pairwise parameter correlation heatmap revealing extensive compensatory structure; strong negative correlations (e.g., extrinsic Xase k_{cat} vs. prothrombinase k_{cat}, $r = -0.81$; Xa→IIa vs. IIa→Va, $r = -0.93$) explain why individual parameters can be poorly recovered while model predictions remain accurate. (c) Patient-specific predictions for Factor VIII deficiency (hemophilia A) at 100%, 30%, and 5% of nominal FVIII levels; ensemble prediction bands capture the dose-dependent reduction in peak thrombin, and true trajectories (black dashed) fall within all three bands. (d) TGA feature accuracy at held-out conditions: ensemble-predicted lag time, peak thrombin, and ETP normalized to true values, with 95% CIs; black crosses indicate the true value is covered by the interval, red crosses indicate it is not.

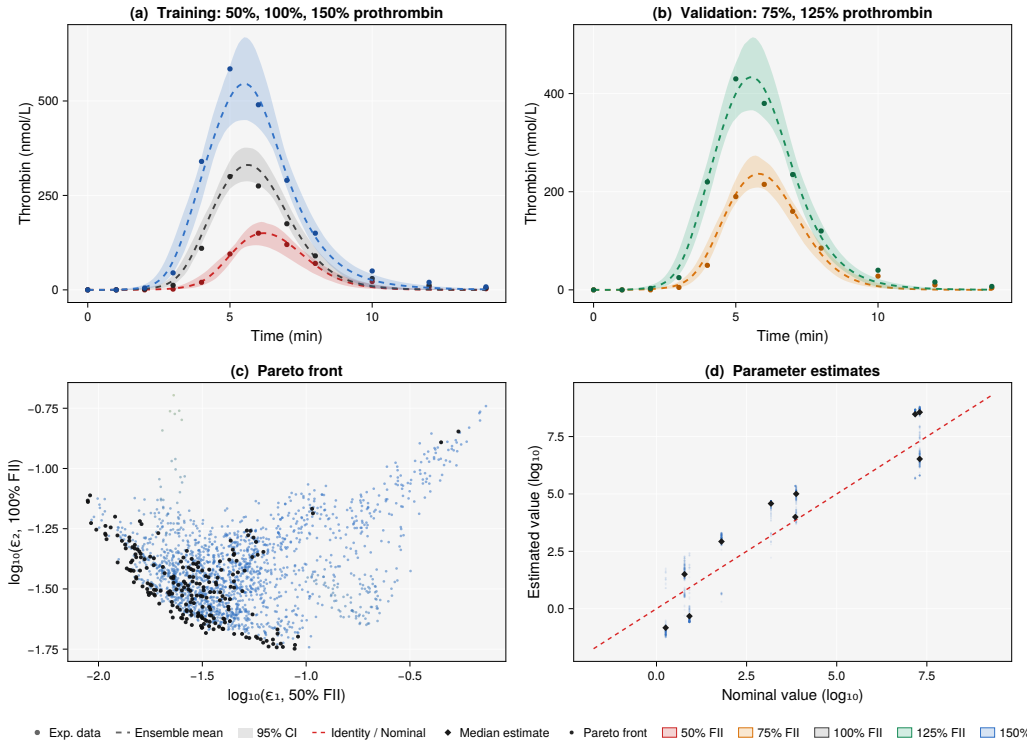

Figure 4: Ensemble estimation fitted to experimental thrombin generation data from Butenas et al. [23]. Prothrombin was varied from 50% to 150% of its mean plasma concentration in a reconstituted synthetic plasma system initiated by 5 pmol/L TF–VIIa. (a) Training fits at 50% (red), 100% (gray), and 150% (blue) prothrombin; the ensemble captures the experimental data at low and normal prothrombin levels but underestimates the peak at 150%, revealing a model–data tension. (b) Held-out validation at 75% (amber) and 125% (teal) prothrombin; the ensemble predicts peak thrombin to within 1–10% at conditions never used during training. (c) Pareto front projection showing trade-offs among the three training objectives. (d) Parameter estimates versus nominal literature values; deviations reflect compensatory adjustments to fit experimental data with an approximate model.

Supplementary Material

S1. Algorithm pseudocode

This section provides complete pseudocode for the four algorithmic components of ParetoEnsembles.jl. We begin with the main simulated annealing loop, which orchestrates the optimization, and then present the subroutines it relies on: the strict dominance test that underlies all ranking decisions, the incremental rank update that keeps per-iteration cost linear, and the multi-chain wrapper that parallelizes the search.

Main algorithm.. The core of ParetoEnsembles.jl is a Pareto simulated annealing loop (Algorithm S1) that maintains an archive of evaluated solutions and their Pareto ranks. At each temperature level, candidates are generated by perturbing the current best parameter vector, and the archive is updated incrementally via RankInsert. Accepted candidates trigger a pruning step that removes highly dominated solutions (rank $\geq R_{\text{cutoff}}$) and enforces a hard archive size cap, followed by a full re-rank of the smaller remaining set. Rejected candidates are immediately removed and the rank array is restored from a snapshot, the *pop-on-reject* strategy described in the main text, preventing unbounded archive growth between accepted moves.

Algorithm S1 ParetoEnsembles: Pareto simulated annealing with incremental ranking

Require: User callbacks: Objective, Neighbor, Accept, Cool
Require: Initial parameter vector $\mathbf{x}_0$; hyperparameters N_{iter}, R_{cutoff}, $T_{\min}$, $n_{\max}$
Ensure: Archive of solutions with objective values, parameters, and Pareto ranks

```
x_best ← x_0
A ← {(x_0, Objective(x_0))};   R ← [0];   T ← 1

while T > T_min do
    for t = 1, ..., N_iter do
        x' ← Neighbor(x_best);   f' ← Objective(x')
        R_saved ← copy(R)
        Append (x', f') to A;   RankInsert(A, R)
        if Accept(R, T) > U(0,1) then
            Prune: A ← {(x_i, f_i) ∈ A | R_i < R_cutoff}
            R ← FullRank(A)
            if |A| > n_max then
                Keep only the n_max solutions with lowest R_i
                R ← FullRank(A)
            end if
            x_best ← x'
        else
            Remove (x', f') from A;   R ← R_saved
        end if
    end for
    T ← Cool(T)
end while
R ← FullRank(A)
return A, R
```

Strict dominance test.. The pairwise dominance check (Algorithm S2) determines whether one solution strictly dominates another by iterating over all m objectives. The test returns TRUE only if solution j is no worse than i on

every objective and strictly better on at least one, which is the strict Pareto dominance relation that corrects the weak dominance used in the original POETs implementation. An early-exit condition terminates the loop as soon as j is found to be worse than i on any single objective, avoiding unnecessary comparisons.

Algorithm S2 StrictlyDominates($\mathbf{f}_j, \mathbf{f}_i, m$): Does solution j strictly dominate solution i?

Require: Objective vectors $\mathbf{f}_j, \mathbf{f}_i \in \mathbb{R}^m$ (lower is better)
Ensure: TRUE if $\mathbf{f}_j \prec \mathbf{f}_i$, FALSE otherwise

```
strictly_better ← FALSE
for k = 1, ..., m do
    if f_{j,k} > f_{i,k} then
        return FALSE                          ▷ j is worse in objective k
    else if f_{j,k} < f_{i,k} then
        strictly_better ← TRUE                ▷ j better in ≥ 1 objective
    end if
end for
return strictly_better
```

Incremental rank update.. When a single candidate is appended to the archive, recomputing all pairwise dominance relationships would cost $O(n^2m)$. The RankInsert procedure (Algorithm S3) exploits the fact that only one solution changed: it performs a single pass over the existing archive, testing dominance in both directions between the new solution and each incumbent. Existing ranks are incremented where the new solution dominates an incumbent, and the new solution's rank is accumulated from the number of incumbents that dominate it. This reduces the per-iteration cost to $O(nm)$, with a full $O(n'^2m)$ re-rank deferred to the pruning step where the archive is smaller.

Algorithm S3 RankInsert($\mathcal{A}, \mathbf{R}$): Incremental rank update after appending one solution

Require: Archive $\mathcal{A} = \{(\mathbf{x}_1, \mathbf{f}_1), \ldots, (\mathbf{x}_n, \mathbf{f}_n)\}$ where $(\mathbf{x}_n, \mathbf{f}_n)$ is new
Require: Rank array $\mathbf{R} = [R_1, \ldots, R_{n-1}]$ valid for solutions $1, \ldots, n-1$
Ensure: Updated $\mathbf{R} = [R_1, \ldots, R_n]$ valid for the full archive

```
R_new ← 0
for i = 1, ..., n − 1 do
    if StrictlyDominates(f_n, f_i, m) then
        R_i ← R_i + 1                          ▷ new solution dominates i
    end if
    if StrictlyDominates(f_i, f_n, m) then
        R_new ← R_new + 1                      ▷ i dominates new solution
    end if
end for
Append R_new to R
```

Multi-chain parallel execution.. Because simulated annealing is inherently sequential within a single chain, the most effective parallelization strategy is to run C independent chains from different starting points and merge the results (Algorithm S4). Each chain receives its own deterministic random number generator seeded from a base seed and the chain index, ensuring full reproducibility. After all chains complete, their archives are concatenated and a final full ranking is performed on the merged set, producing a combined ensemble with broader coverage of the trade-off surface than any individual chain can achieve.

Algorithm S4 ParetoEnsemblesParallel: Multi-chain parallel execution

Require: Initial states $\{\mathbf{x}_0^{(1)}, \ldots, \mathbf{x}_0^{(C)}\}$; base seed s
Ensure: Merged archive with Pareto ranks

```
for c = 1, ..., C in parallel do
    rng_c ← Seed(s, c)
    (A_c, R_c) ← ParetoEnsembles(x_0^(c), rng_c)
end for
A ← A_1 ∪ ··· ∪ A_C
R ← FullRank(A)
return A, R
```

S2. Software design

ParetoEnsembles.jl is a pure Julia package with no dependencies beyond the standard library module `Random`, and its public API consists of five functions (Table S1). The primary entry point, `estimate_ensemble`, runs a single Pareto SA chain and returns the retained objective values, parameter vectors, and Pareto ranks; its parallel counterpart, `estimate_ensemble_parallel`, runs C independent chains concurrently, merges their archives, and re-ranks the combined set. A standalone `rank_function` computes Pareto ranks for an arbitrary $m \times n$ objective matrix and can be used independently of the optimization loop, while `hypervolume` computes the hypervolume indicator for a 2D front using a sweep-line algorithm in $O(n \log n)$ time and `pareto_front` extracts the rank-zero subset from the archive for downstream analysis.

The algorithm is entirely defined by four user-supplied callback functions (Table S2): an objective function that maps a parameter vector $\mathbf{x} \in \mathbb{R}^d$ to an $m \times 1$ array of objective values, a neighbor function that perturbs a given parameter vector to generate a candidate, an acceptance probability function that returns a scalar probability from the rank array and temperature, and a cooling function that maps the current temperature to a reduced value. Beyond these four callbacks, the behavior of `estimate_ensemble` is controlled by keyword arguments including the pruning threshold `rank_cutoff` (default 5), the number of candidates per temperature level `maximum_number_of_iterations` (default 20), the stopping temperature `temperature_min` (default 10^{-4}), and the hard archive size cap `maximum_archive_size` (default 1000). When `trace=true`, the algorithm records convergence diagnostics (temperature, archive size, and hypervol-

ume) at each cooling step, returning a fourth element in the output tuple that allows users to monitor convergence and assess whether the annealing schedule was sufficient.

Table S1: Public API of ParetoEnsembles.jl.

Function	**Description**
`estimate_ensemble`	Run a single Pareto SA chain. Returns objective values, parameter vectors, and Pareto ranks for the retained archive.
`estimate_ensemble_parallel`	Run C independent chains in parallel (one per starting point), merge archives, and re-rank.
`rank_function`	Compute Pareto ranks for an $m \times n$ objective matrix.
`hypervolume`	Compute the hypervolume indicator for a 2D Pareto front given a reference point.
`pareto_front`	Extract Pareto-optimal (rank = 0) solutions from the archive.

S3. Parallel execution

Two opportunities for parallelism arise from the structure of the algorithm. First, the outer loop of both the full ranking and the incremental RankInsert procedure iterates independently over archive solutions, so setting `parallel_evaluation=true` dispatches to threaded variants that distribute these independent iterations across available threads using Julia's

Table S2: User-supplied callback functions.

Callback	**Signature**	**Purpose**
`objective_function`	$(\mathbf{x}) \to \mathbb{R}^{m\times 1}$	Evaluate m objectives at parameter vector $\mathbf{x}$
`neighbor_function`	$(\mathbf{x}) \to \mathbb{R}^{d}$	Generate a candidate by perturbing $\mathbf{x}$
`acceptance_probability_function`	$(\mathbf{R}, T) \to [0, 1]$	Acceptance probability given ranks and temperature
`cooling_function`	$(T) \to T'$	Annealing schedule

`Threads.@threads` construct, with an atomic accumulator used for R_{new} in the incremental case since multiple threads contribute to it. Second, and more significantly, because simulated annealing is inherently sequential within a single chain, the most effective parallelization strategy is to run C independent chains from different starting points and merge the resulting archives (Algorithm S4); each chain receives its own deterministic random number generator seeded from a base seed and the chain index, ensuring reproducibility, and after all chains complete, the archives are concatenated and a final full ranking is performed on the merged set.

To quantify the benefit of multi-chain parallelism, we measured wall clock time for the cell-free ensemble estimation (ten chains, $N_{\text{iter}} = 50$) as a function of the number of Julia threads (Table S3). Each configuration was run five times and the median time is reported; the speedup is approximately linear up to four threads (3.1×) and reaches 4.1× at eight threads, with

diminishing returns due to load imbalance when ten chains are distributed across eight threads and the serial cost of the final archive merge and re-ranking step.

Table S3: Multi-chain parallel scaling on the cell-free gene expression model (ten chains, $N_{\text{iter}} = 50$, median of five runs).

Threads	**Wall clock (s)**	**Speedup**
1	18.52	1.0×
2	9.38	2.0×
4	5.94	3.1×
8	4.51	4.1×

S4. Standard benchmarks

We validated ParetoEnsembles.jl on two standard multiobjective benchmarks: the constrained Binh–Korn problem [19] (Equation (D.1)) and the unconstrained Fonseca–Fleming problem [20] with $d = 3$ decision variables (Equation (D.2)). Constraints in the Binh–Korn problem are enforced via quadratic penalty terms added to the objective values, and bound constraints are handled by clamping in the neighbor function. We ran both benchmarks using ten parallel chains with $N_{\text{iter}} = 60$, $R_{\text{cutoff}} = 12$, and $\alpha = 0.95$, and the resulting Pareto fronts in objective space recover the characteristic trade-off curves for both problems, with a visible cloud of near-optimal solutions surrounding the non-dominated front (Figure S1a,b). The computed front for the Fonseca–Fleming case lies directly on the theoretical curve, confirming that the algorithm converges to the correct solution set, and in parameter space the Binh–Korn Pareto-optimal decision vectors trace a curved manifold from the origin toward $(5, 3)$ while the Fonseca–Fleming solutions cluster along the diagonal $x_1 \approx x_2$ as expected from the symmetric structure of the objectives (Figure S1c,d). Comparing a single chain against ten parallel chains on the Binh–Korn problem illustrates the benefit of multi-chain execution (Figure S2): the single chain recovers the trade-off curve but, because every retained solution is non-dominated within that chain's archive, the rank array contains only rank-zero entries, whereas the merged multi-chain

run produces a denser front with broader coverage of the feasible region.

$$\begin{aligned} \min_{\mathbf{x}} \quad & f_1 = 4x_1^2 + 4x_2^2, \qquad f_2 = (x_1-5)^2 + (x_2-5)^2 \\ \text{s.t.} \quad & (x_1-5)^2 + x_2^2 \le 25, \qquad (x_1-8)^2 + (x_2-3)^2 \ge 7.7 \\ & 0 \le x_1 \le 5, \quad 0 \le x_2 \le 3. \end{aligned} \tag{D.1}$$

$$f_1(\mathbf{x}) = 1 - \exp\left(-\sum_{i=1}^{d}\left(x_i - \tfrac{1}{\sqrt{d}}\right)^2\right), \qquad f_2(\mathbf{x}) = 1 - \exp\left(-\sum_{i=1}^{d}\left(x_i + \tfrac{1}{\sqrt{d}}\right)^2\right). \tag{D.2}$$

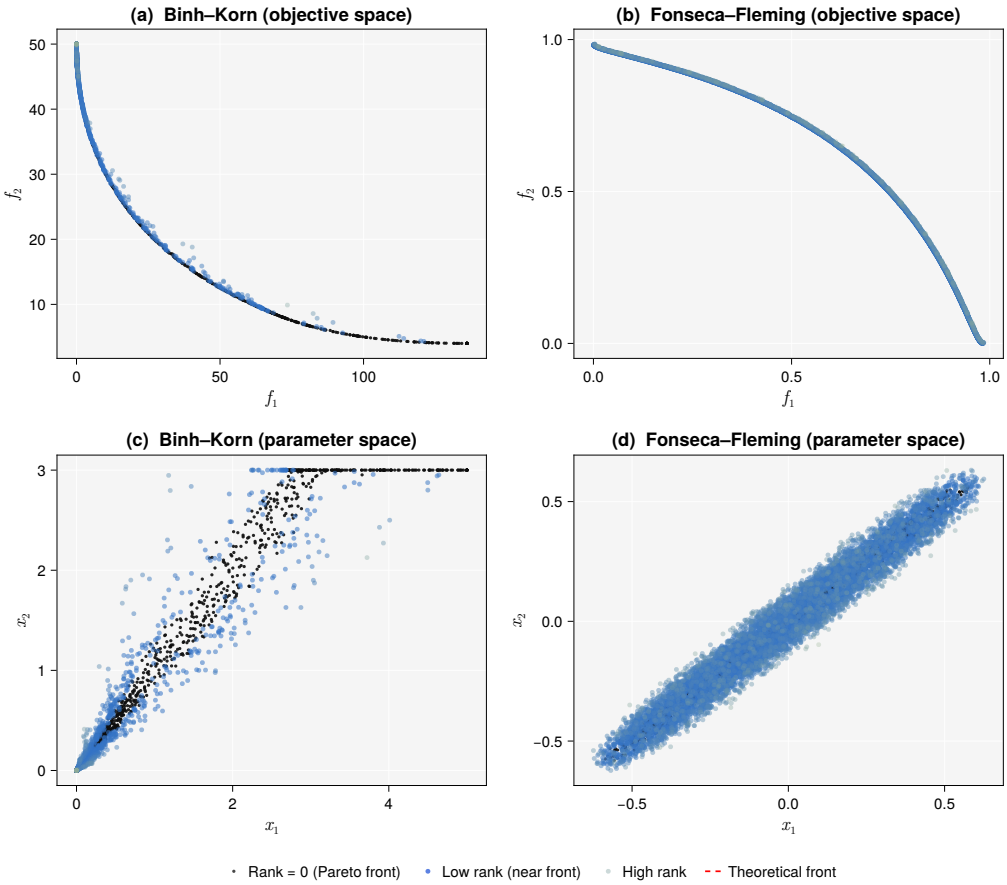

Figure S1: Benchmark results for (a,c) Binh–Korn and (b,d) Fonseca–Fleming ($d = 3$). Top row: objective space; bottom row: parameter-space projections (x_1 vs. x_2). Non-dominated solutions (rank = 0, dark) define the Pareto front, while near-optimal solutions (blue, colored by rank) form a cloud around it representing the retained ensemble. The dashed red curve in (b) is the theoretical Pareto front. Ten parallel chains were used with $R_{\text{cutoff}} = 12$, $N_{\text{iter}} = 60$, and $\alpha = 0.95$.

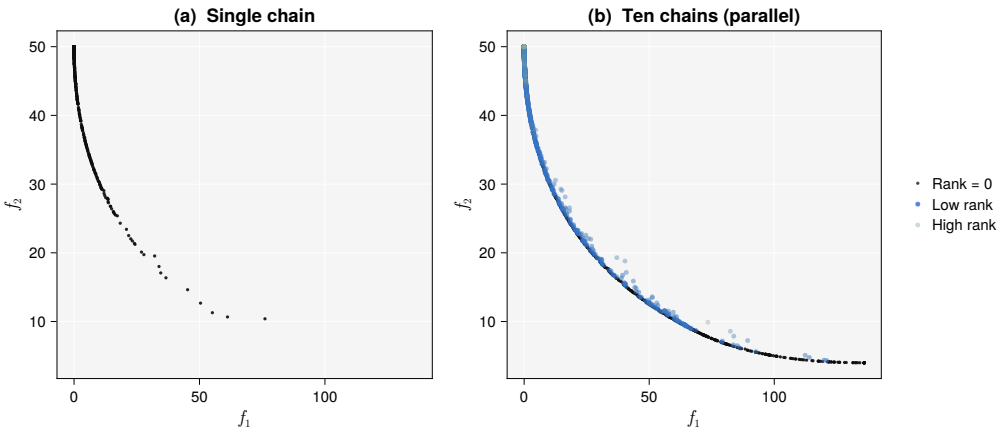

Figure S2: Comparison of (a) single-chain and (b) ten-chain parallel execution on the Binh–Korn problem. Both panels share the same axis limits. The single chain yields a sparse archive of non-dominated solutions, while the merged multi-chain archive provides denser front coverage and a visible cloud of near-optimal solutions (blue) surrounding the front.

S5. Comparison with NSGA-II

To position ParetoEnsembles.jl relative to population-based evolutionary methods, we compared it against NSGA-II [5] as implemented in Metaheuristics.jl [14] on both benchmark problems, matching the function evaluation budget at approximately 110,000 evaluations per solver (ParetoEnsembles: 10 chains, $N_{\text{iter}} = 60$, $\alpha = 0.95$; NSGA-II: population 200, 550 generations). Five independent replicates were run for each solver, and the median hypervolume indicator (HV) and inverted generational distance (IGD) are reported in Table S4, with IGD computed against the analytical Pareto front for the Fonseca–Fleming problem and against a densely sampled reference front from a high-budget run for Binh–Korn. ParetoEnsembles.jl achieved comparable or slightly higher hypervolume than NSGA-II on both problems and substantially lower IGD (5× lower on Binh–Korn and 7× lower on Fonseca–Fleming), indicating a closer approximation to the true Pareto front (Figure S3), while also producing much denser fronts with 2,000–6,000 non-dominated solutions compared with 200 for NSGA-II, reflecting its design as an ensemble generator that characterizes the full cloud of near-optimal solutions rather than merely approximating the front boundary. NSGA-II was 50–150× faster in wall time, consistent with its population-parallel evaluation strategy, but this speed advantage comes at the cost of a sparser representation of the trade-off surface that may be insufficient for downstream uncertainty analysis.

Table S4: Comparison of ParetoEnsembles.jl and NSGA-II on benchmark problems (median over 5 replicates). Higher HV and lower IGD are better. Both solvers used ∼110,000 function evaluations.

Problem	**Solver**	**HV**	**IGD**	**Front size**	**Time (s)**
Binh–Korn	ParetoEnsembles	**8035**	**0.014**	6172	77
	NSGA-II	8027	0.071	200	0.5
Fonseca–Fleming	ParetoEnsembles	**0.44**	**0.0004**	2051	3.1
	NSGA-II	0.44	0.003	200	0.5

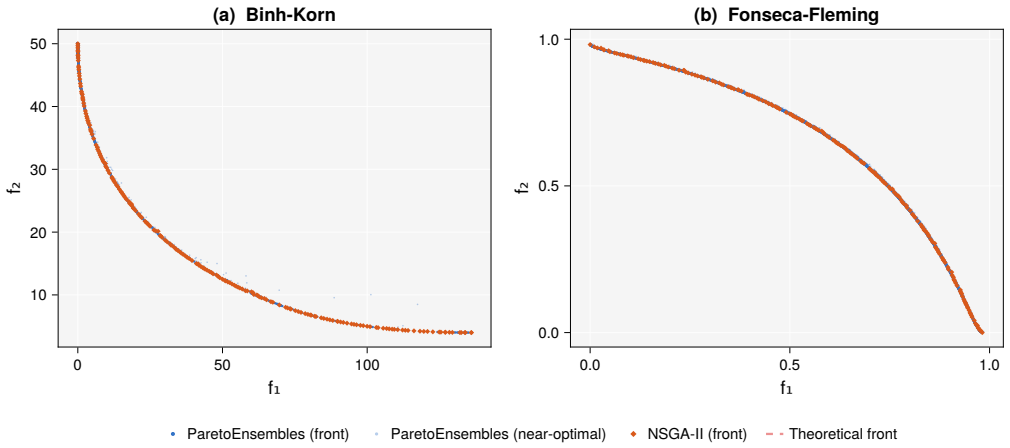

Figure S3: Comparison of ParetoEnsembles.jl (left) and NSGA-II (right) on (a,b) Binh–Korn and (c,d) Fonseca–Fleming at matched evaluation budgets (∼110,000). ParetoEnsembles.jl retains a cloud of near-optimal solutions (blue) in addition to the Pareto front (black), while NSGA-II returns only the non-dominated front. The dashed red curve in (c,d) is the theoretical Fonseca–Fleming front.

S6. Hyperparameter sensitivity

We assessed the sensitivity of ParetoEnsembles.jl to three key hyperparameters (rank cutoff R_{cutoff}, cooling rate α, and iterations per temperature N_{iter}) by sweeping each while holding the others at default values ($R_{\text{cutoff}} = 8$, $\alpha = 0.90$, $N_{\text{iter}} = 40$) on the Binh–Korn benchmark with five chains and five replicates per configuration (Figure S4). The hypervolume indicator was robust across all tested ranges: rank cutoff values from 2 to 12 produced nearly identical results, with a modest decline at $R_{\text{cutoff}} = 20$ where the archive retains many low-quality solutions; faster cooling ($\alpha = 0.80$) performed comparably to slower cooling ($\alpha = 0.95$), with the latter showing slightly more variability; and even 10 iterations per temperature level produced competitive fronts, though 20 or more reduced inter-replicate variance. These results suggest that the default hyperparameters provide a reasonable starting point for most problems, and that users need not invest significant effort in tuning, a practical advantage for the systems biology applications where ParetoEnsembles.jl is primarily intended to be used.

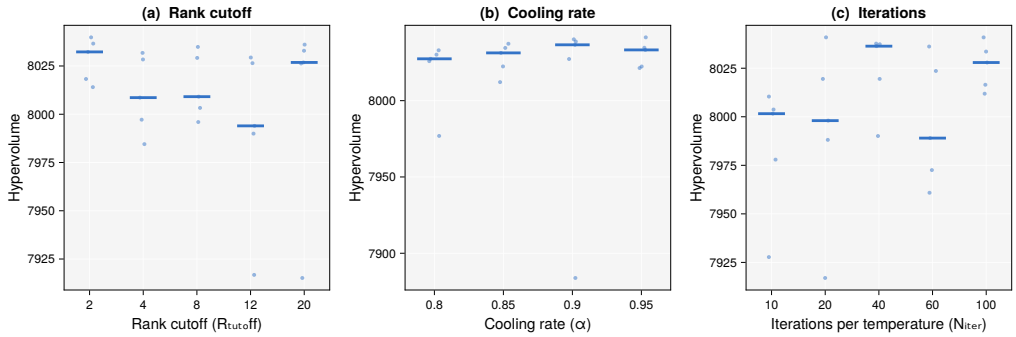

Figure S4: Hyperparameter sensitivity on the Binh–Korn benchmark. Median hypervolume (line) with interquartile range (shaded band) over 5 replicates. (a) Rank cutoff R_{cutoff}, (b) cooling rate α, (c) iterations per temperature N_{iter}. Default values for the other two parameters are held constant in each sweep.

S7. Robustness to model misspecification

The coagulation results in the main text use the same model to generate and fit the data, so the fitting model is exactly correct by construction, a setting sometimes called an "inverse crime" in the inverse problems literature. To assess how the ensemble approach performs when this assumption is violated, we repeated the coagulation study with a deliberately misspecified model: six rate constants that are *not* among the ten estimated parameters were perturbed by ±30% from their nominal values before generating the synthetic training data, so that no parameter vector in the fitting model's search space can perfectly reproduce the data. The perturbed parameters span the initiation (TF=VIIa+VII→VIIa), binding (extrinsic Xase and IX activation K_m), assembly (prothrombinase on-rate), and inhibition (Xa+TFPI and mIIa+ATIII) pathways, ensuring that the misspecification affects multiple phases of the coagulation cascade.

Despite the model error, the ensemble still produced thrombin trajectories that closely tracked the training data at all three TF concentrations (Figure S5a), demonstrating that the ten estimated parameters absorbed much of the misspecification through compensatory adjustments. At the held-out validation conditions (10, 20, and 30 pM TF), the ensemble predicted peak thrombin to within 4–6% of the true values (Figure S5b), comparable to the 6–7% error observed with the correct model in the main text. However, trajectory-level coverage degraded substantially: the 95% prediction intervals covered the true trajectory at only 31–48% of time points, compared with higher coverage under the correct model, reflecting the ensemble's inability to account for structural model error through parametric uncertainty alone.

The pattern of TGA feature accuracy also shifted under misspecification (Figure S5c). Lag time, which was well-covered in the correct-model study, showed a systematic overprediction of 5–9% with the true value falling outside the 95% interval at some conditions, while ETP, which was biased under the correct model, was now predicted to within 0.3–2.5% with good coverage. Peak thrombin remained biased by 4–6%, similar to the correct-model case. These shifts reflect the fact that the estimated parameters compensate for the misspecified fixed parameters differently depending on which aspect of the thrombin curve is being evaluated, and they highlight a fundamental limitation of parametric uncertainty quantification: the ensemble can only explore the space of parameter vectors within a given model structure, and when that structure is itself approximate, the resulting prediction intervals may be miscalibrated even though the point predictions remain accurate. Parameter recovery (Figure S5d) showed wider scatter and larger biases than the correct-model case, as expected since the "true" parameter values are no longer optimal under the misspecified model.

These results confirm that the ensemble approach is robust to moderate model misspecification, with predictions degrading gracefully rather than catastrophically, while honestly revealing the boundary between parametric uncertainty, which the ensemble captures, and structural uncertainty, which requires either model expansion or explicit model-error terms to address.

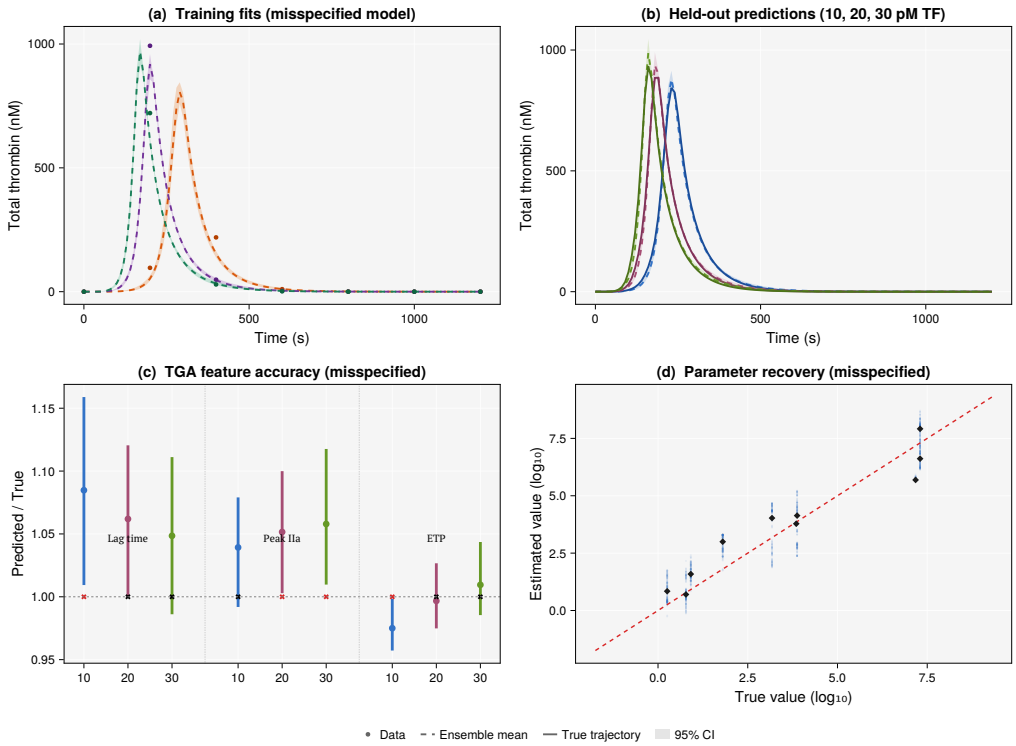

Figure S5: Model misspecification study: six fixed rate constants perturbed by ±30% before generating training data, then fitted with the nominal model. (a) Training fits at 5, 15, 25 pM TF; the misspecified model still tracks the data. (b) Held-out predictions at 10, 20, 30 pM TF; ensemble means (dashed) are within 4–6% of true trajectories (solid) but 95% CIs (shaded) do not fully bracket the truth. (c) TGA feature accuracy at held-out conditions, normalized to true values; the bias pattern shifts relative to the correct-model case. (d) Parameter recovery shows wider scatter and larger biases than the correct model, as expected when the fitting model is structurally approximate.

S8. Code examples

Listing 1: Binh–Korn benchmark with ParetoEnsembles.jl.

```julia
using ParetoEnsembles

function objective_function(x)
    f = zeros(2, 1)
    f[1] = 4.0 * x[1]^2 + 4.0 * x[2]^2
    f[2] = (x[1] - 5)^2 + (x[2] - 5)^2
    # penalty for constraint violations
    lambda = 100.0
    v1 = 25 - (x[1] - 5)^2 - x[2]^2
    v2 = (x[1] - 8)^2 + (x[2] - 3)^2 - 7.7
    f[1] += lambda * (min(0, v1))^2
    f[2] += lambda * (min(0, v2))^2
    return f
end

neighbor_function(x) = clamp.(
    x .* (1 .+ 0.05 * randn(length(x))),
    [0.0, 0.0], [5.0, 3.0])

acceptance_probability_function(R, T) =
    exp(-R[end] / T)

cooling_function(T) = 0.9 * T

```

```julia
(EC, PC, RA) = estimate_ensemble(
    objective_function, neighbor_function,
    acceptance_probability_function, cooling_function,
    [2.5, 1.5]; rank_cutoff=4.0,
    maximum_number_of_iterations=40, show_trace=false)

pareto_idx = findall(RA .== 0)
```

Listing 2: Multi-chain parallel execution on the Binh–Korn problem (requires `julia -t4`).

```julia
using ParetoEnsembles

# ... (same callbacks as Listing 1) ...

initial_states = [
    [2.5, 1.5], [0.5, 2.5],
    [4.0, 0.5], [1.0, 1.0]
]

(EC, PC, RA) = estimate_ensemble_parallel(
    objective_function, neighbor_function,
    acceptance_probability_function, cooling_function,
    initial_states;
    rank_cutoff=4.0, maximum_number_of_iterations=40,
    show_trace=false, rng_seed=42)

println("Solutions: ", size(EC, 2))
println("Pareto-optimal: ", count(RA .== 0))
```